UTEC
Universidad Tecnológica

Patrón Composite

Descripción General

El patrón Composite es un patrón estructural que permite crear estructuras de objetos en forma de árbol para representar jerarquías de "parte-todo". Este patrón trata a los objetos individuales y a las composiciones de objetos de manera uniforme, es decir, permite que los objetos compuestos de otros objetos se traten como si fueran objetos individuales.

Este patrón es muy útil cuando necesitas trabajar con una estructura que puede contener objetos individuales (hojas) y también otros objetos compuestos (nodos). El objetivo es que tanto las hojas como los nodos se manejen de manera uniforme a través de una interfaz común.

Aplicaciones Comunes

- **Sistemas de archivos**: Donde los directorios pueden contener archivos u otros directorios.
- **Interfaz gráfica de usuario (GUI)**: Donde los componentes gráficos pueden contener otros componentes.
- **Documentos estructurados**: Como XML o HTML, donde los elementos pueden contener otros elementos o texto.

Estructura del Patrón

1. **Componente (Component)**: Es una interfaz o clase abstracta que declara las operaciones comunes que se deben implementar en hojas y nodos compuestos.
2. **Hoja (Leaf)**: Implementa las operaciones declaradas en el componente, representando los objetos individuales en la estructura.
3. **Compuesto (Composite)**: Implementa las operaciones del componente y tiene la capacidad de contener otros componentes, ya sean hojas u otros compuestos.

Ventajas

- **Simplicidad**: Permite tratar objetos individuales y compuestos de manera uniforme.
- **Flexibilidad**: Facilita la adición de nuevos tipos de componentes (hojas o compuestos) sin cambiar el código existente.
- **Mantenimiento**: Mejora la organización del código al manejar las jerarquías de objetos de manera más estructurada.

UTEC
Universidad Tecnológica

Ejemplo en Java

Vamos a implementar un ejemplo de un sistema de archivos donde los archivos y carpetas se manejan mediante el patrón Composite.

```
import java.util.ArrayList;
import java.util.List;

// Componente
interface FileSystemComponent {
    void showDetails();
}

// Hoja: Representa un archivo
class File implements FileSystemComponent {
    private String name;

    public File(String name) {
        this.name = name;
    }

    @Override
    public void showDetails() {
        System.out.println("File: " + name);
    }
}

// Compuesto: Representa una carpeta que puede contener archivos y otras
carpetas
class Directory implements FileSystemComponent {
    private String name;
    private List<FileSystemComponent> components = new ArrayList<>();

    public Directory(String name) {
        this.name = name;
    }

    public void addComponent(FileSystemComponent component) {
        components.add(component);
    }
```

UTEC
Universidad Tecnológica

```
    public void removeComponent(FileSystemComponent component) {
        components.remove(component);
    }

    @Override
    public void showDetails() {
        System.out.println("Directory: " + name);
        for (FileSystemComponent component : components) {
            component.showDetails();
        }
    }
}

// Ejemplo de uso
public class CompositePatternExample {
    public static void main(String[] args) {
        File file1 = new File("Document1.txt");
        File file2 = new File("Image1.png");
        File file3 = new File("Video1.mp4");

        Directory directory1 = new Directory("Documents");
        Directory directory2 = new Directory("Media");
        Directory rootDirectory = new Directory("Root");

        directory1.addComponent(file1);
        directory2.addComponent(file2);
        directory2.addComponent(file3);
        rootDirectory.addComponent(directory1);
        rootDirectory.addComponent(directory2);

        rootDirectory.showDetails();
    }
}
```

En este ejemplo, hemos implementado un sistema de archivos donde `File` representa archivos individuales y `Directory` representa carpetas que pueden contener tanto archivos como otras carpetas. Ambos implementan la interfaz `FileSystemComponent`, lo que permite tratarlos de manera uniforme. En el método `main`, creamos varios archivos y

directorios, y luego los organizamos en una estructura de árbol. Finalmente, mostramos los detalles de toda la estructura comenzando desde el directorio raíz.

Conclusión

El patrón Composite es muy útil cuando trabajas con estructuras jerárquicas complejas. Permite manejar de manera uniforme tanto objetos individuales como compuestos, lo que simplifica el diseño y mantenimiento de tu código. Este patrón es ampliamente utilizado en aplicaciones que requieren manipular estructuras como sistemas de archivos, interfaces gráficas y documentos estructurados.